# CMP205 Application Design

## Comic Book Store Project

### Application Design Report

**[Connor Murphy 1903299]**

# Design (40%)

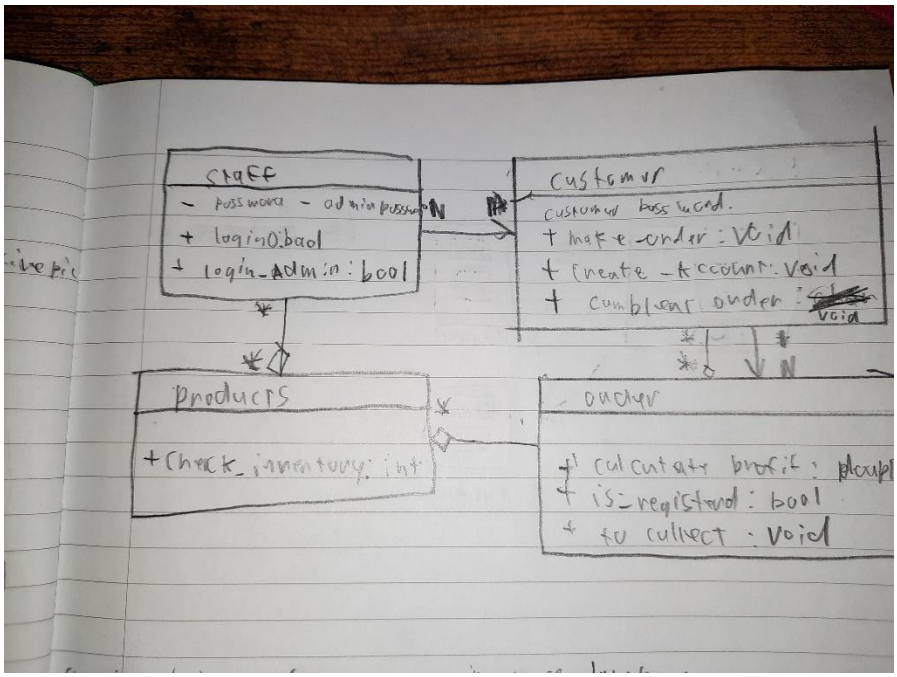
## Analysis

The design of my classes and overall program are very similar to that of my database, with the database tables being replaced with their own unique classes with the variables being the data from the database. The main aspect I focused on when making the classes, however, are the functions that used the variables from each table as needed.  Originally, I planned on having the functions of a login verification to be in the respective classes (staff login in their staff class, customer login in the customer class.). However, the finished project has the functions in their own private class and the functions use the auto generated classes variable from an entity framework method of connecting to a database. By doing this the associations of the classes heavily become one sided on the classes that contain the functions.

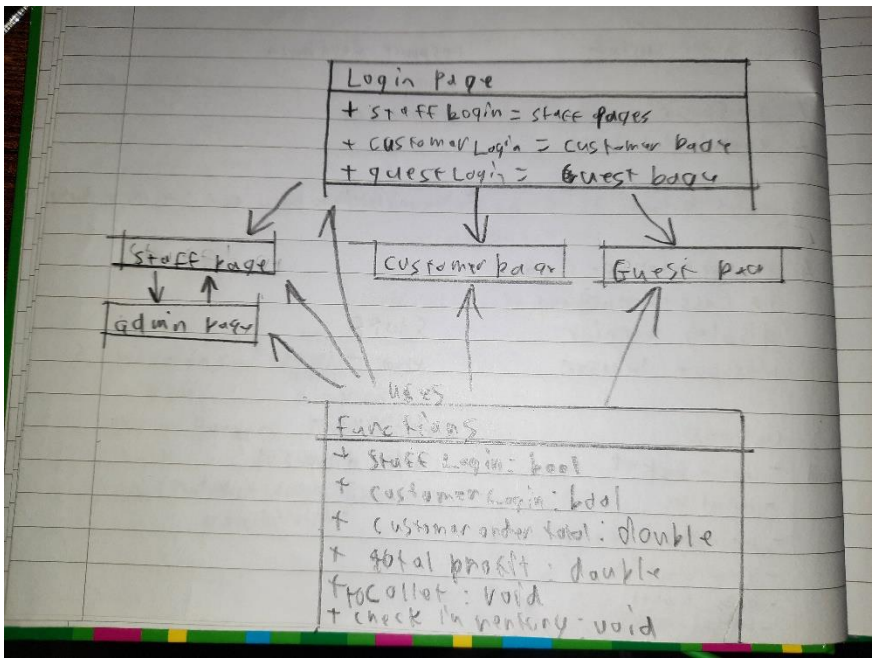
Since this project is making a windows app, the classes of the project also contain forms that count as their own classes. I handled this dichotomy by separating the database classes/ functions and the winform forms classes. By doing this, I focused the functions that manipulate the forms contained to their respective classes. To show this I made two class diagrams. The winform classes naturally have no association but when I made functions that load different forms, I made a meaningful connection.

## Model

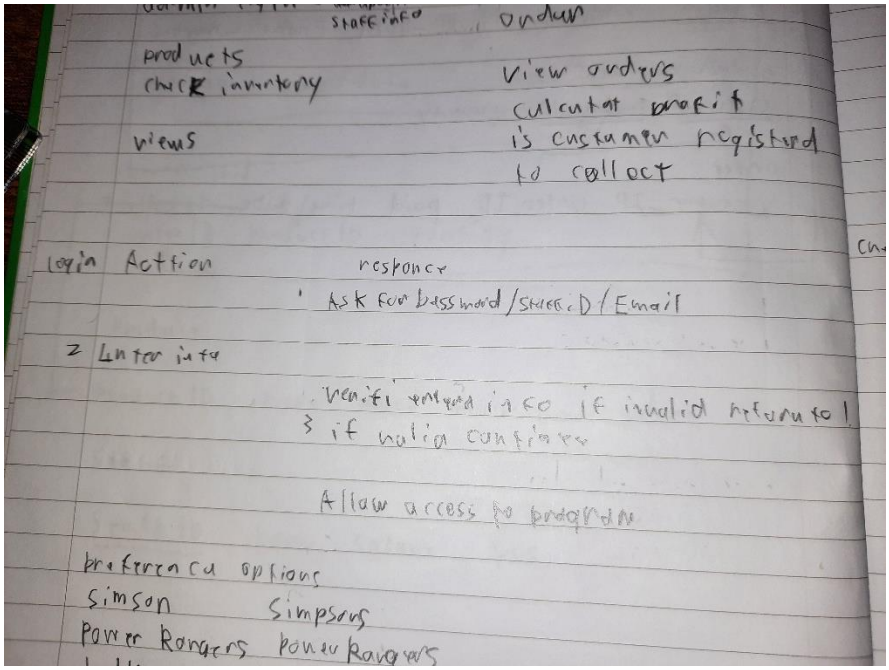
First class diagram



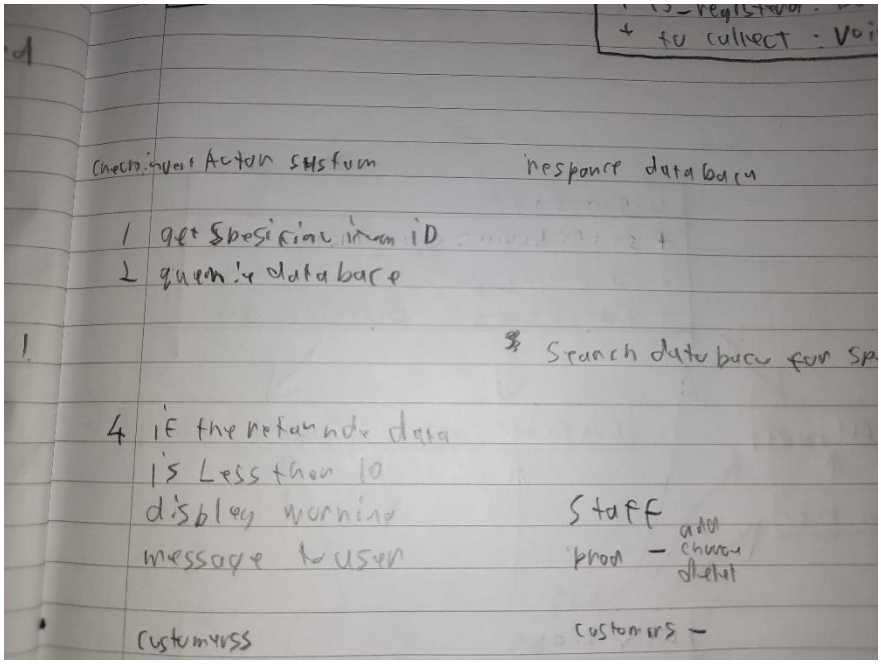

Last class diagram




Login Use case




Checkinfo Use case diagram



# Implementation (40%)

## C# Source Code and Demo

I am submitting

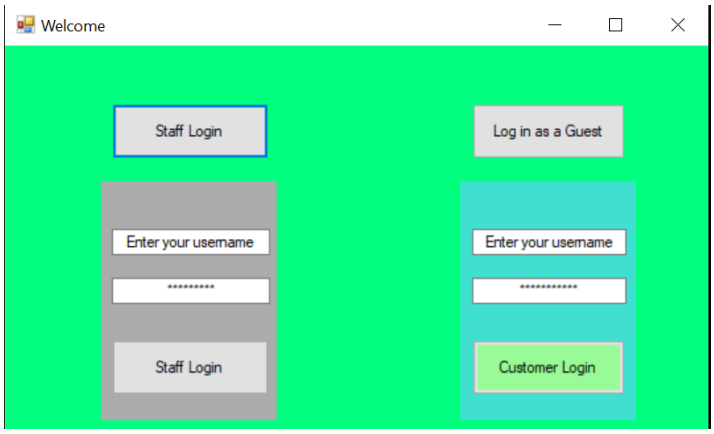
☐ .cs files **yes**

☐ .exe file **yes**

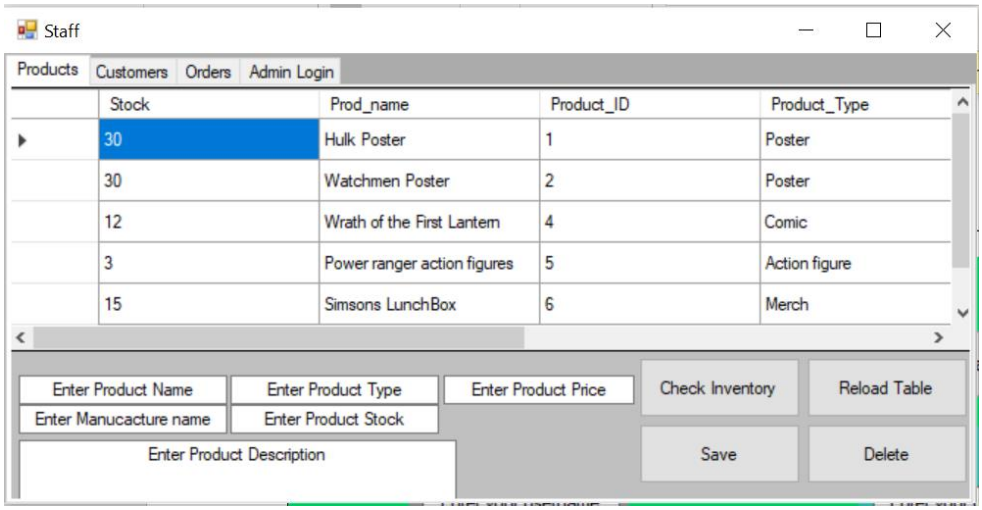
☐ Video demo (or link to video) **yes**

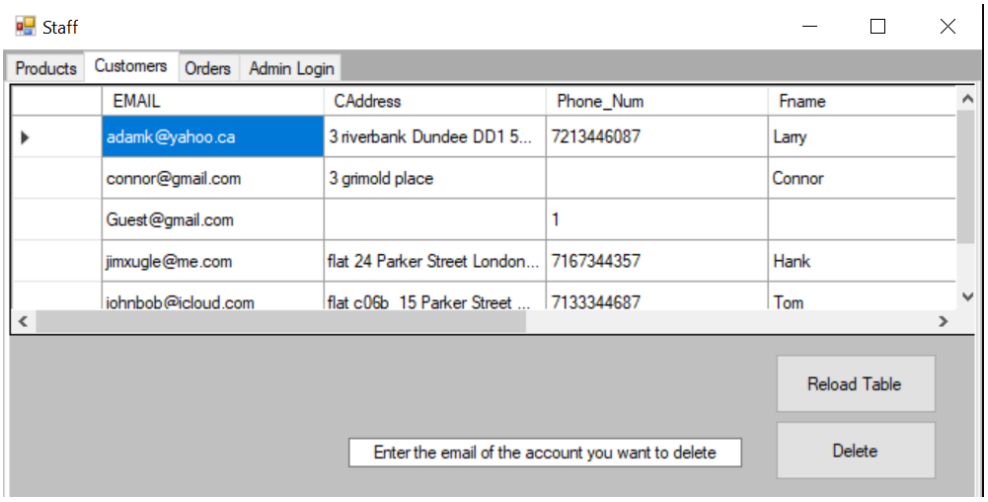
## User Interface

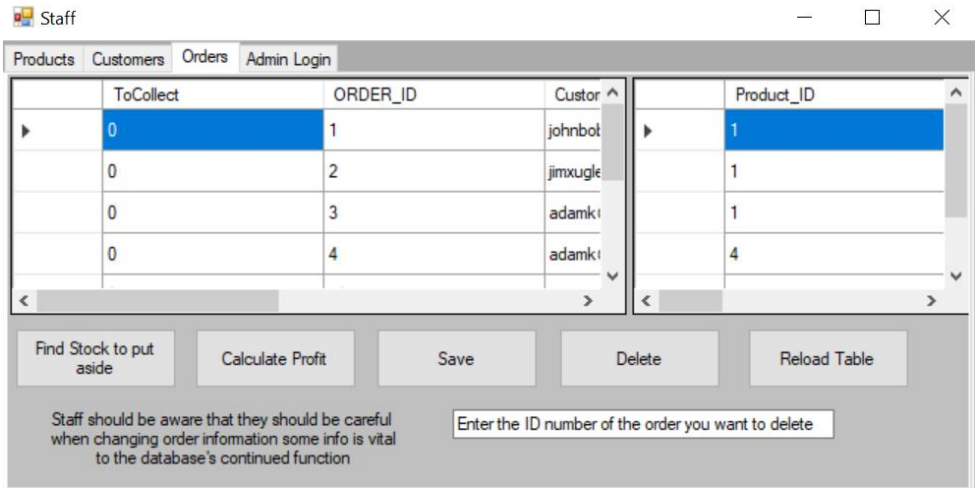
Login page staff login id blue customer



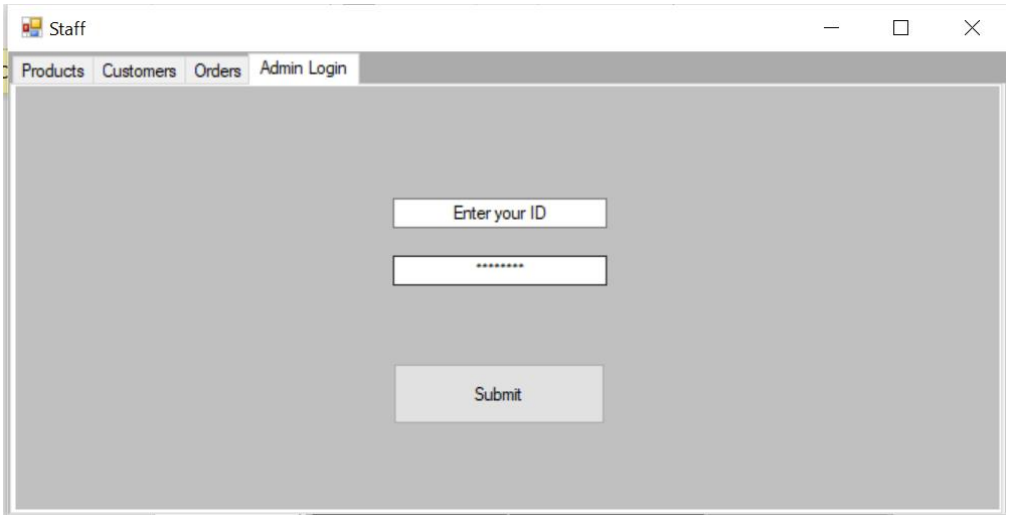

Staff pages



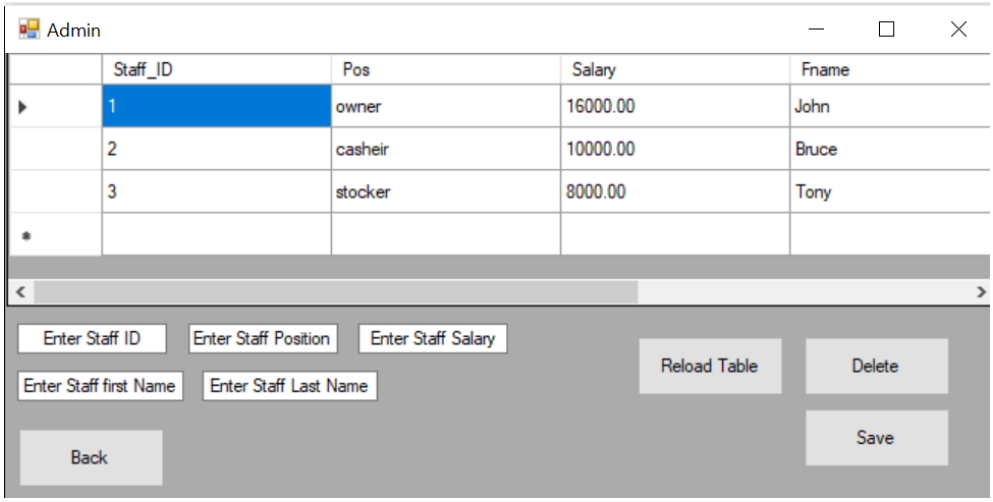


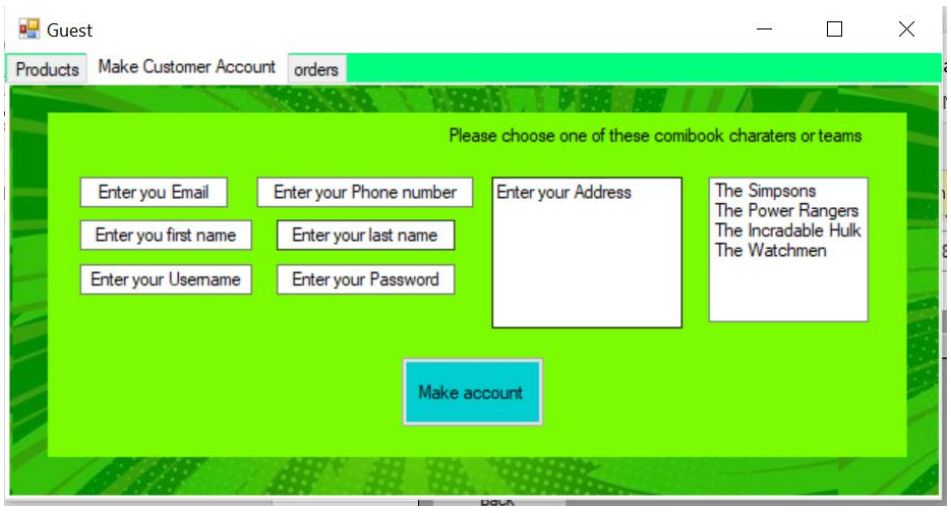

Admin Login id 1 password green



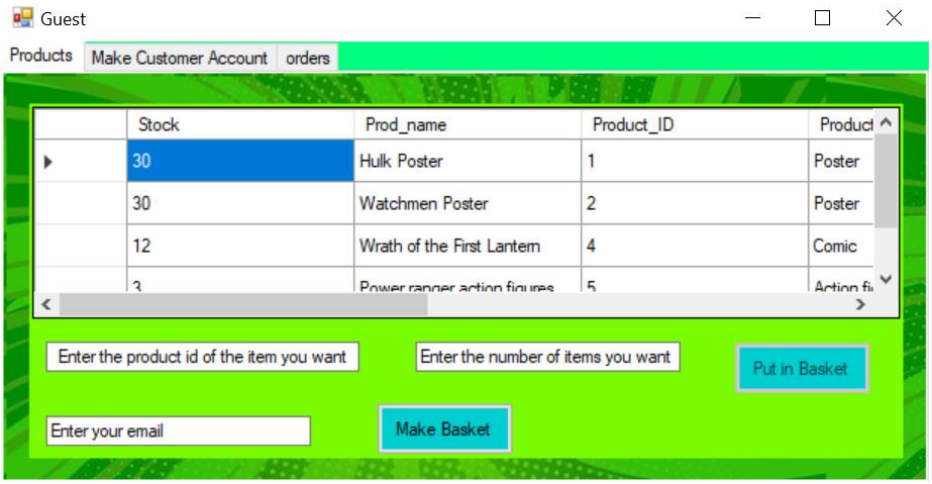

Admin page



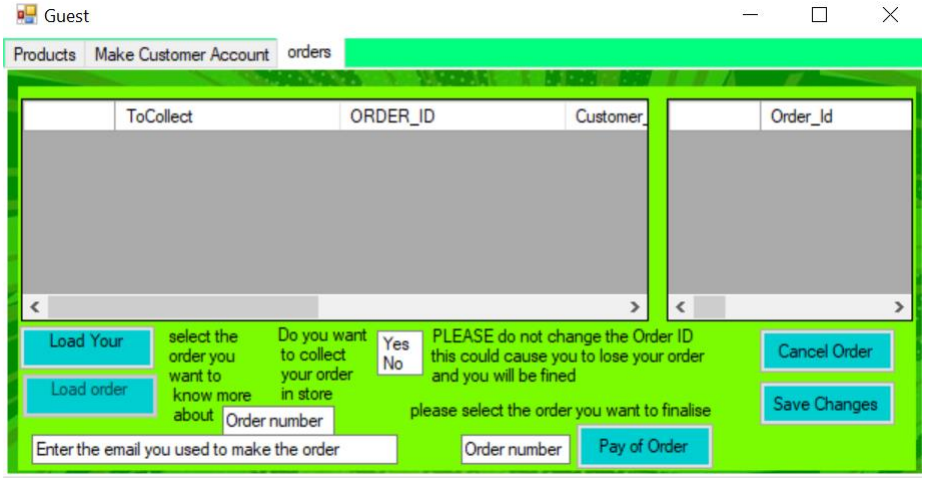

Guest create account page



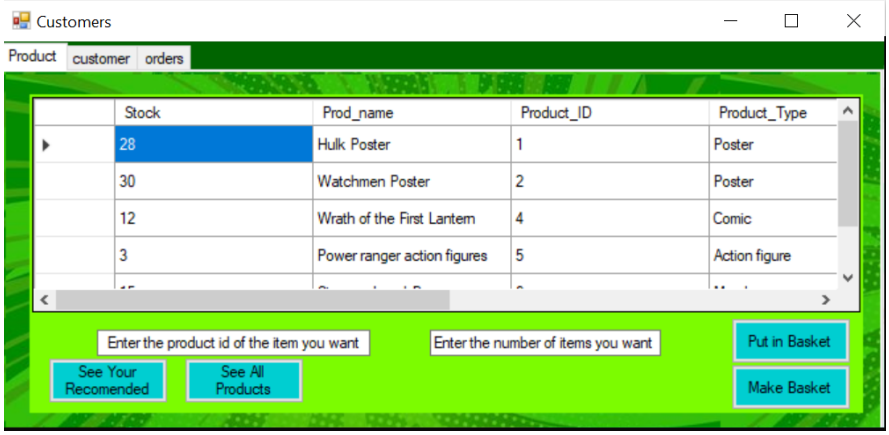

Guest product page

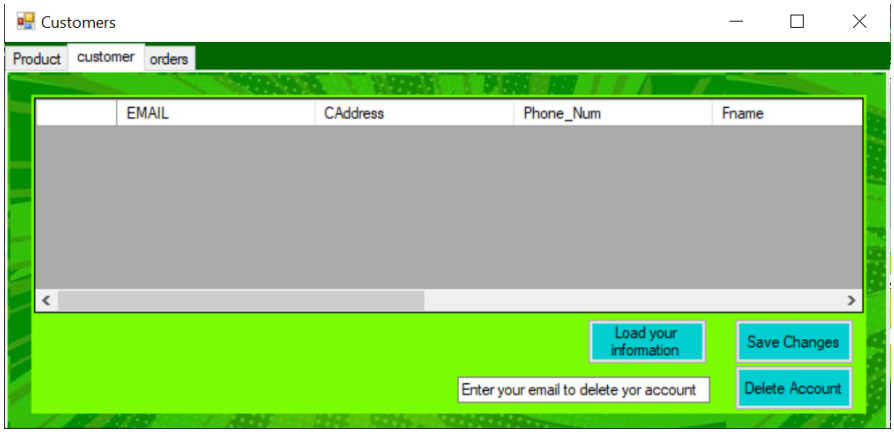


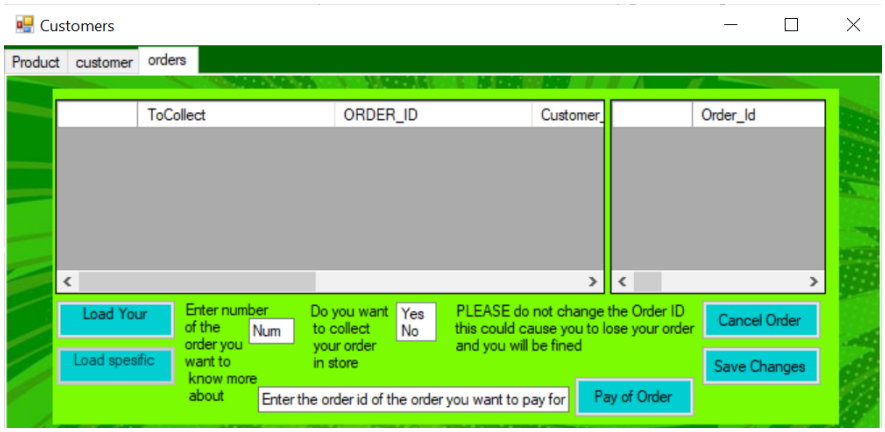

Guest order page




Customer pages






# Reflection (20%)

## Development Methodology

It was decided that the waterfall methodology will be used for this project. This method was chosen because, despite it's slow and methodical approach there would be no time for multiple iterations to be worked through, as there would be only one person working on this project. Using this method would allow the creation of the best version of the application in the short time frame that was given.  During the requirements stage it was discussed what the application needed to achieve. To do this a list of functional and Non-functional requirements were written up as well as a database to hold all the necessary information in a logical fashion. At the design stage the database was made to the initial design. For the software design conceptual classes were written up to accommodate all the required functional needs. For the complex required functions use cases were made and

simplified for smooth implementation. For the implementation stage the real practical work started by making the main WinForms (pages) of the application. These were made to include all the buttons and input needed for functionality and streamlined user experience. The finished database was connected to the relevant tables. Due to the time constraints for the project the testing stage was added to the implementation stage, meaning that after a function was made it was vigorously tested for any errors or for any incorrect logic. By using this method, it ensured that every functional requirement was met and that returning an incomplete final product would be avoided.

## Evaluation

The initial design decision was to use entity framework to manipulate the premade database and use the auto generated classes to add functions that uses their respective tables and rows. This initial design meets most OOP design principles. The data (info from the database) and functions are encapsulated and abstracted respectively. In this design inheritance was not needed since it would add needless complexity. Also, most functions will be overloaded by their demand needs.

However, when the implication stage was reached it was found that if a programmer adds a function to one of the auto generated classes, it meant that when the function was called it would only have access to a single row not the entire table. This obviously would not work for the functional requirements. So, to fix this issue it was decided to have high level background functions such as password validation and profit calculations. These would be in a special class on their own. This separation of classes also led to the decision to leave winforms manipulation functions in their respective classes. This naturally led to a design that has high levels of decoupling and abstraction.

## Personal Reflections

Whilst I worked through this module it has occurred to me, that this module has shown me how I view work as a student. To explain, for me Unit 1 was a simple and enjoyable task maybe because I have had experience making databases before. I understood and liked the process of sitting down and planning/discussing how to make the database followed by the action of making said database. Unit 2 however, has shown me that I don't handle setbacks well. For one reason or another I had to start from scratch on 4 separate occasions. I have

also been reminded that I have a bad tendency of over complicate problems to the point that I run in to a wall. One example is that of a function that calculates the total profit of the store. The way I designed my database meant that the variables I needed to access, were spread across 3 tables. I first tried to write a program that accessed all three tables at once. I was not until I spoke to my tutor that I realised by adding one column I could simplify the problem greatly.

## References (if you use external material owned by others)

[refer to Abertay's referencing guide on how to cite different material.]

Customer and guest background image address
https://t3.ftcdn.net/jpg/02/27/92/58/360_F_227925845_6faxUiBzocszqGsmdVsu0B83srJKalvX.jpg